PROMPT ENGINEERING PLAYGROUND

Full Project Overview & Development Plan

Project type	Full-stack web application
Difficulty	Beginner to intermediate
Core stack	React · Python · FastAPI · SQLite · LLM API
Focus	Prompt experimentation and REST API development
Excluded	Docker and CI/CD

A practical project guide covering features, architecture, database design, REST APIs, implementation phases, testing, and local setup.

Prompt Engineering Playground

Full Project Overview and Development Plan

Project type: Full-stack web application

Difficulty: Beginner to intermediate

Primary focus: Prompt engineering, REST API development, LLM integration, and database operations

Suggested stack: React, Python, FastAPI, SQLite, and an LLM API

Deployment: Optional simple application hosting

Scope: No Docker or CI/CD required

---

1. Project Overview

Prompt Engineering Playground is a web application that helps users create, test, compare, evaluate, and manage prompts for large language models (LLMs).

A user writes a prompt, selects a prompting technique, supplies any required input, and sends it to an LLM through the backend. The application displays the generated response and allows the user to save the prompt, record feedback, and compare different prompt versions.

The main purpose is to provide a simple workspace for experimenting with prompt wording and techniques while keeping a history of experiments.

Problem Statement

When experimenting with prompts, users often edit prompts in separate documents or chat windows. It can be difficult to remember which version produced a useful result, compare outputs, or reuse previous prompts.

This application provides one place to create prompts, run experiments, save results, and compare versions.

Objectives

- Provide a simple interface for writing and editing prompts.
- Demonstrate common prompting techniques.
- Connect to an LLM using a backend API.
- Save prompts and generated responses.
- Compare outputs from different prompt versions.
- Allow users to rate or comment on results.
- Demonstrate practical full-stack and REST API development.

---

2. Target Users and Use Cases

The application is suitable for students, developers, content creators, and people learning how to work with LLMs.

Example use cases:

– A student tests different prompts to explain a technical topic.

– A developer compares two instructions for generating code explanations.

– A content creator saves reusable prompts for drafting content.

– A user revisits an earlier experiment and checks its response and rating.

---

3. Core Features

3.1 Prompt Editor

- Enter a prompt title and prompt text.
- Edit the prompt before running it.
- Add optional context or input text.
- Select an LLM model from the models configured by the application.
- Submit the prompt and view the response.

3.2 Prompting Techniques

Provide selectable prompt templates or presets:

- **Zero-shot:** Ask the model to perform a task without examples.
- **Few-shot:** Include a small number of input/output examples.
- **Role-based:** Give the model a role and task context.
- **Structured output:** Request a defined format, such as JSON or bullet points.
- **Step-by-step task instructions:** Ask for a clear, ordered explanation or procedure.

These are prompt templates, not guarantees that a model will behave consistently.

3.3 AI Response Generation

- Send the prompt from the React interface to FastAPI.
- Validate the request on the backend.
- Call the configured LLM provider.
- Return the generated answer to the frontend.
- Display useful errors if the provider request fails.

The LLM provider and model should be configurable. Keep API keys in backend environment variables; never expose secrets in React code or commit them to Git.

3.4 Prompt History

- Save prompt text, technique, model name, response, and timestamp.
- View previously executed prompts.

- Reopen a saved prompt for editing or reuse.
- Delete a history entry when required.

3.5 Prompt Comparison

- Select two saved prompt runs.
- Display their prompt text and generated responses side by side.
- Let the user compare clarity, relevance, completeness, and format.
- Allow the user to add a rating or written note.

Comparison is initially user-led. The application should not claim that one response is objectively better unless a defined evaluation method supports that conclusion.

3.6 Prompt Evaluation

For the first version, use a simple user rating:

- Relevance: 1–5
- Clarity: 1–5
- Completeness: 1–5
- Optional feedback note

An optional later enhancement can use a separate LLM evaluation request. Clearly label automated evaluations as model-generated estimates, not objective facts.

3.7 Prompt Templates

- Save reusable prompts with a title and category.
- Browse and select templates.
- Copy a template into the editor.
- Edit a template without changing the original unless the user saves the update.

---

4. Technology Stack

Layer	Technology	Purpose
Frontend	React	User interface
Backend	Python + FastAPI	REST API and application logic
Database	SQLite	Store prompts, runs, and ratings
ORM	SQLAlchemy (optional)	Database access and models
LLM	An API provider selected by the developer	Generate responses
API testing	Postman or Bruno	Test backend endpoints
Code editor	VS Code	Development
Version control	Git and GitHub	Source code management

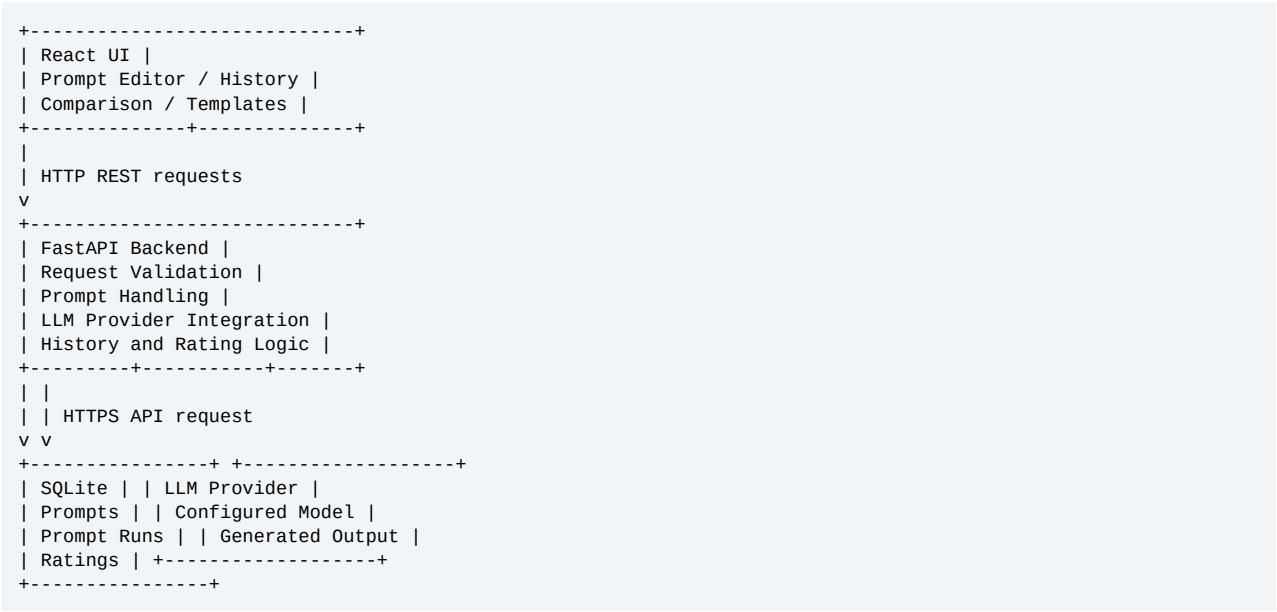
Why this stack?

- Python and FastAPI keep backend development approachable.
- React provides a component-based interface.
- SQLite avoids the need to configure a database server for the first version.
- An LLM API allows the project to focus on prompt experimentation rather than training a model.

No Docker or CI/CD pipeline is required for this project.

---

5. High-Level Architecture



Request flow

– The user writes a prompt in React.

– React sends the prompt and selected technique to FastAPI.

– FastAPI validates the input and prepares the final model request.

– FastAPI calls the configured LLM provider.

– The provider returns a response.

– FastAPI stores the prompt run in SQLite.

– FastAPI returns the response and run identifier to React.

– The user can rate the result or compare it with another run.

---

6. Suggested Application Screens

6.1 Dashboard

Show:

- Total saved prompts.
- Total prompt runs.
- Recently used prompts.
- Quick action to create a prompt.

6.2 Prompt Playground

Show:

- Prompt title.
- Prompt editor.
- Optional context/input field.
- Technique selector.
- Model selector.
- Run Prompt button.
- Generated response panel.
- Save or rate actions.

6.3 Prompt History

Show:

- Prompt title.
- Technique.
- Model.
- Run date.
- Short response preview.
- Open, reuse, compare, and delete actions.

6.4 Comparison Page

Show two selected runs side by side:

- Original prompt.
- Technique used.
- Model name.
- Generated response.
- User ratings and notes.

6.5 Template Library

Show saved templates by category, with actions to preview, use, edit, and delete.

---

7. Database Design

A simple first version can use three main tables. If login is not required, omit the Users table and keep the application single-user.

7.1 Users (optional)

Column	Type	Description
id	INTEGER, primary key	User identifier
username	TEXT	Display name
email	TEXT, unique	Email address
password_hash	TEXT	Hashed password; never store plain passwords

7.2 Prompts

Column	Type	Description
id	INTEGER, primary key	Prompt identifier
title	TEXT	Prompt title
prompt_text	TEXT	Prompt content
technique	TEXT	Selected technique
category	TEXT	Optional category
created_at	DATETIME	Creation time
updated_at	DATETIME	Last update time

7.3 PromptRuns

Column	Type	Description
id	INTEGER, primary key	Run identifier
prompt_id	INTEGER, foreign key	Related prompt
model_name	TEXT	Model used
input_text	TEXT	Optional task input
response_text	TEXT	Generated response
relevance_rating	INTEGER	User rating from 1–5
clarity_rating	INTEGER	User rating from 1–5
completeness_rating	INTEGER	User rating from 1–5
feedback_note	TEXT	Optional user feedback
created_at	DATETIME	Run timestamp

7.4 PromptTemplates (optional)

Column	Type	Description
id	INTEGER, primary key	Template identifier
title	TEXT	Template title

Column	Type	Description
category	TEXT	Template category
template_text	TEXT	Reusable prompt
created_at	DATETIME	Creation time

Relationships:

- One prompt can have many prompt runs.
- A prompt run belongs to one prompt.
- Templates are reusable and can be copied into a new prompt.

---

8. REST API Design

Suggested endpoints:

Method	Endpoint	Purpose
POST	`/api/prompts`	Create a prompt
GET	`/api/prompts`	List saved prompts
GET	`/api/prompts/{id}`	Get one prompt
PUT	`/api/prompts/{id}`	Update a prompt
DELETE	`/api/prompts/{id}`	Delete a prompt
POST	`/api/prompts/{id}/run`	Execute a prompt using the LLM
GET	`/api/runs`	List prompt runs
GET	`/api/runs/{id}`	Get a run and its response
POST	`/api/runs/{id}/rating`	Save ratings and feedback
GET	`/api/templates`	List templates
POST	`/api/templates`	Create a template
DELETE	`/api/templates/{id}`	Delete a template

Example request: create a prompt

```
{
"title": "Explain machine learning",
"prompt_text": "Explain machine learning to a beginner using a simple example.",
"technique": "zero-shot",
"category": "Education"
}
```

Example request: run a prompt

```
{
"model_name": "configured-model",
"input_text": "The intended audience is a first-year student."
}
```

The backend should combine the saved prompt and optional input, then send the resulting request to the configured LLM provider.

---

9. Recommended Folder Structure

```
prompt-engineering-playground/
|
|-- backend/
| |-- app/
| | |-- main.py
| | |-- database.py
| | |-- models/
| | | |-- prompt.py
| | | |-- prompt_run.py
| | |-- schemas/
| | | |-- prompt.py
| | |-- routes/
| | | |-- prompts.py
| | | |-- runs.py
| | | |-- templates.py
| | |-- services/
| | |-- llm_service.py
| | |-- prompt_service.py
| |-- tests/
| |-- requirements.txt
| |-- .env.example
|
|-- frontend/
| |-- src/
| | |-- components/
| | |-- pages/
| | |-- services/
| | |-- App.jsx
| | |-- main.jsx
| |-- package.json
|
|-- README.md
|-- .gitignore
```

This is a suggested structure. It can be simplified further for an initial prototype.

---

10. Development Phases

Phase 1 — Project Setup

- Create the project folders.
- Set up a Python virtual environment.
- Install FastAPI and required packages.
- Create the React application.
- Configure SQLite.

Deliverable: React frontend and FastAPI backend run locally.

Phase 2 — Prompt Management

- Create the prompt editor.
- Implement create, view, update, and delete operations.

- Store prompts in SQLite.
- Test the REST endpoints.

Deliverable: Users can manage prompts without LLM integration.

Phase 3 — LLM Integration

- Select an LLM API provider.
- Store its API key securely in the backend environment.
- Implement the LLM service.
- Add prompt execution and response display.
- Handle API errors, timeouts, and rate limits.

Deliverable: Users can submit prompts and receive AI responses.

Phase 4 — History and Comparison

- Save each prompt run.
- Display prompt history.
- Allow users to reopen and reuse prompts.
- Build a side-by-side comparison page.

Deliverable: Users can review and compare previous experiments.

Phase 5 — Evaluation and Templates

- Add manual ratings and feedback.
- Create a reusable template library.
- Add basic filters and search.

Deliverable: A more complete prompt experimentation workspace.

Phase 6 — Testing and Documentation

- Test API success and failure cases.
- Test database operations.
- Check frontend/backend integration.
- Write setup instructions and API documentation.
- Prepare screenshots for the GitHub README.

Deliverable: A tested, documented application ready for demonstration.

---

11. Testing Plan

Test the following scenarios:

Area	Test case
Prompt creation	Save a valid prompt
Validation	Reject an empty prompt
Prompt editing	Update prompt text
Prompt deletion	Delete an existing prompt
LLM integration	Receive a response for a valid request
LLM error handling	Show a useful message if the provider is unavailable
History	Retrieve previous runs
Comparison	Display two different runs
Rating	Save ratings within the 1–5 range
Security	Ensure API keys are not returned to the frontend

Use FastAPI's testing support with pytest for backend tests. Test the LLM service using a mocked provider response so that tests do not require a paid API call.

---

12. Security and Reliability Considerations

- Keep LLM API keys in backend environment variables.
- Never commit `.env` files or secrets to GitHub.
- Validate request fields and limit prompt/input sizes.
- Handle provider timeouts, rate limits, and invalid responses.
- Avoid logging API keys or sensitive user input.
- If accounts are introduced, hash passwords and enforce authorization.
- Make it clear that LLM responses can be inaccurate and should be reviewed.

---

13. Minimum Viable Product (MVP)

Build these features first:

– Prompt editor.

– Technique selector with zero-shot, few-shot, and role-based presets.

– LLM API integration.

– Response display.

– Save prompts and prompt history.

– Basic side-by-side comparison.

Do not begin with authentication, advanced automated scoring, multiple providers, or complex analytics. Add those only after the MVP works.

---

14. Future Enhancements

- Support multiple LLM providers.
- Add prompt versioning.
- Add token usage and estimated cost tracking.
- Add export to TXT or CSV.
- Add optional model-assisted prompt evaluation.
- Add user accounts and private prompt libraries.
- Add prompt categories and tags.

---

15. Expected Final Outcome

The finished application will provide a single workspace where users can create prompts, select prompting techniques, send requests to an LLM, review generated responses, save experiments, rate outputs, and compare prompt versions.

The project demonstrates practical knowledge of:

- Python application development.
- FastAPI and REST API design.
- React frontend development.
- SQLite and relational data modelling.
- External API integration.
- Input validation and error handling.
- Automated testing.
- Prompt engineering and LLM application development.

Final project title: Prompt Engineering Playground

GitHub description: An interactive full-stack application to create, test, compare, evaluate, and manage AI prompts using different prompting techniques and LLM integration.

---

16. Suggested Local Setup

Backend

```
cd backend
python -m venv .venv
```

Activate the environment:

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Install dependencies:

```
pip install fastapi uvicorn sqlalchemy python-dotenv pytest httpx
```

Run the API:

```
uvicorn app.main:app --reload
```

Open the interactive API documentation at:

`http://127.0.0.1:8000/docs`

Frontend

Create and run the React application using your preferred React setup. Configure the frontend to call the local FastAPI backend.

Environment example

```
LLM_API_KEY=replace_with_your_key
LLM_MODEL=replace_with_supported_model
```

Use the environment variable names expected by your chosen provider integration. Do not put real credentials in `.env.example`.

Scope reminder: This is an application-development project, not an LLM model-training project. It uses an existing model through an API. Docker and CI/CD are intentionally excluded.